

TC 자동화 시스템 개요

프로젝트별 TC(테스트 케이스) 설계부터 Playwright 자동 실행, 결함 관리까지 이어지는 QA 자동화 시스템입니다. 터미널/IDE에서 Claude Code를 직접 쓰거나(경로 A), 로컬 웹 대시보드에서 채팅·버튼으로 조작할 수 있습니다(경로 B). 2026-08-24 Slack 봇("큐돌이") 운영을 종료하고 이 두 경로로 완전히 전환했습니다. 아래는 이 시스템을 처음 보는 동료에게 설명하기 위한 구조·프로세스·사용법 정리입니다.

인터페이스: 터미널 / IDE 직접 세션 + 대시보드 (:4000)

저장소: **tc-automation** (Git, 단일 저장소) 대시보드 구동: **Windows** 예약 작업 (**\$4U**) 상시

최종 갱신: **2026-09-29**

목차

- | | |
|----------------------------|------------------------------|
| 01 실행 경로 & 토큰 사용 영역 | 02 TC 생성·실행·결함관리 프로세스 |
| 03 대시보드 기능 지도 | 04 문서(MD) 관리 위치 |
| 05 알려진 문제·제약 | 06 사용자 워크플로우 |
| 07 한눈에 보는 용어 | |

01 실행 경로 & 토큰 사용 영역

Claude를 실제로 호출(=비용 발생)하는 기능과, 코드만으로 결정론적으로 처리되어 호출하지 않는(zero-token) 기능을 나눴습니다. TC 설계처럼 실제 추론이 필요한 작업만 Claude를 쓰고, 이미 확정된 데이터를 반복 처리하는 작업은 대시보드가 코드로 직접 처리합니다.

기능	경로	토큰
----	----	----

TC 신규 생성·수정 (Phase 0~4, 정책서/URL 분석)	A: 터미널·IDE 세션 / B: 대시보드 채팅 패널	● 사용
결함관리 중 자유 형식 질문 ("이 결함 원인이 뭐야?")	A 또는 B 채팅	● 사용
TC 업로드(엑셀/JSON) 완료 후 TC 뷰어(HTML) 재생성	B → 완료 즉시 채팅에 자동 요청(백그라운드)	● 사용 (자동 트리거)
TC 엑셀·JSON 업로드 반영 (수정/신규 판정, legacy 아카이브, 버전관리)	B: "+ TC 업로드"	● 미 사용
조건별 테스트 실행 (모듈/시스템/우선순위/수행상태 필터, 또는 TC 뷰어 체크박스)	B: 테스트 실행 큐	● 미 사용
테스트 결과 → TC json / defects.json / results 스냅샷 반영	B: 코드가 직접 처리	● 미 사용
테스트 실행 화면 녹화 & 재생(영상·스크린샷 팝업)	B: 테스트 실행 큐 "단계 재생" 모드	● 미 사용
결함 상태·담당자·이슈링크 변경	B: 결함 관리 표에서 직접 편집	● 미 사용
git add + commit (테스트 실행/업로드 완료 시)	B: 자동	● 미 사용
git push	B: "🔗 Git" 버튼 — 사용자가 직접 클릭해야만 실행	● 미 사용

02 TC 생성 → 실행 → 결함관리 프로세스

프로젝트 온보딩부터 결함 등록까지 이어지는 Phase 0~8 워크플로우(AGENTS.md 13항)입니다. 굵게 표시된 지점은 사람의 "승인"이 있어야만 다음 단계로 넘어갑니다 — 과거 Slack 자동연속 예외는 2026-08-24부로 전면 폐지되어, 어떤 경로로 요청하든 Phase당 1회씩 승인을 받습니다.

_template/을 project/{프로젝트명}/으로 복사 (Policy·SB·Requirements·TC). 단위/통합, 코드 기반/정책기반, URL 여부를 질의해 project.json에 저장 — 이후 요청부터는 재질의하지 않음.

1~3

Phase 1~3 — 근거 확보 & 시나리오 검토표

정책서 분석 또는 URL 관찰(Playwright) → 테스트 범위 산정 → 도출 가능한 전체 시나리오를 있는 그대로 제시

|| 승인 대기

4

Phase 4 — TC 상세 작성 & 산출물 생성

TC HTML/JSON 캐노니컬 파일 생성·갱신(legacy 아카이브 + meta.version/changeHistory). URL 기반이면 Playwright 자동화 코드도 함께 생성(실행은 안 함)

|| 승인 대기

5

Phase 5 — 테스트 실행 (별도 요청 필요)

A: 터미널/IDE 세션이 직접 `npm run playwright test` 실행 / B: 대시보드 테스트 실행 큐로 필터 조합 실행(zero-token, 실시간 로그·영상 녹화 옵션)

6~7

Phase 6~7 — 오류 공유 & 결함 관리

콘솔 에러·4xx/5xx는 자동으로 테스트를 실패시키고 스크린샷·콘솔로그를 확보. defects.json에 tclد 기준 신규/재발생/기존 실패 유지로 자동 판정 후 등록·갱신

8

Phase 8 — 결과 도출 & git 반영

Pass/Fail 요약 리포트 직후 자동 `git add project/{프로젝트명} → commit.push`는 항상 별도로 확인받은 뒤에만 실행(대시보드는 "🔗 Git" 버튼, 세션은 매번 재확인)

03 대시보드 기능 지도

로컬 웹 대시보드(`dashboard/` , Express + `ws` , 포트 4000)에서 프로젝트를 선택하면 아래 기능들을 쓸 수 있습니다. 표시 없는 기능은 전부 zero-token입니다.

 KPI · 결함 · 실행이력

 사이트분석 · TC 생성 (채팅)

프로젝트별 결함 현황, 모듈별 수행율/Pass율, 우선순위 분포, 날짜별 실행 이력 스냅샷을 한 화면에서 확인

"큐돌이"에게 자연어로 TC 생성·수정·결함 문의 —
claude CLI를 헤드리스로 spawn(토큰 사용).
Phase당 승인 절차 동일 적용

▶ 테스트 실행 큐

모듈·시스템·우선순위·수행상태를 조합 선택(0건인 조합은 자동 비활성화) → Playwright 순차 실행 → 결과가 TC/defects.json/results에 자동 반영 + git 자동 커밋. "단계 재생" 모드로 실행 영상도 확인 가능

+ TC 업로드

엑셀(.xlsx) 또는 JSON 파일을 업로드하면 TC ID 기준으로 자동 수정/신규 판정. 같은 ID인데 분류·항목이 다르면 별개 케이스로 분리 생성(오답 방지 포함)

🔍 결함 관리

상태·담당자·이슈링크를 표에서 바로 편집 — 저장 즉시 defects.json에 반영(단, 이 편집 자체는 git 자동 커밋 대상 아님 — 잦은 소소한 변경이라 배치로 커밋)

🔗 Git 상태 · push

origin 대비 몇 커밋 앞서 있는지·최근 커밋 목록을 보여주고, 버튼을 눌러야만 push — 저장소 전체 기준(프로젝트 무관)

📄 Full TC 뷰어 — 체크박스 실행

TC 뷰어(HTML) 목록에서 TC를 체크하고 "선택 TC 테스트 실행" — 대시보드와 같은 세션 쿠키를 공유해 같은 테스트 실행 큐 엔진을 그 자리에서 호출

+ 새 프로젝트 / 관리 / 삭제

_template/ 복사로 신규 생성, project.json 필드 수정, 삭제 시 backup/deleted-projects/에 자동 백업 후 로컬에서만 제거(git 반영은 별도)

04 문서(MD) 관리 위치

2026-08-24 `agents-config` · `slack-bridge` 를 git subtree로 이 저장소에 병합한 뒤로, 사실상 모든 규칙·코드 문서가 하나의 저장소에서 버전관리됩니다 — 과거(v0.2 문서) 가장 큰 리스크였던 "행동 규칙 미버전관리" 문제는 해소되었습니다.

파일

위치

용도

`CLAUDE.md`

저장소 루트

Claude Code가 세션 시작 시 자동 로드하는 진입점 — 저장소 안내 +
AGENTS.md/SKILL.md import

AGENTS.md	agents-config/AGENTS.md	전체 행동 규칙 (TC 스키마, 우선순위, 근거원칙, Phase 0~8, Git 규칙, 결함관리 등)
SKILL.md	agents-config/skills/qa-test-case-generator/SKILL.md	TC 생성 스킬 상세 스펙 (컬럼 정의, HTML 뷰어 스펙 등)
task 스킬 3종 (playwright-automation 등)	.claude/skills/	Claude Code가 자동 인식하는 고정 위치 — 필요할 때만 lazy load
PIPELINE.md / PRD.md / PORTABLE_QA_METHODOLOGY.md	_reference/	설계 배경, 현재 파이프라인 정의(PRD.md 13항), 타 저장소 이식용 일반화 버전
대시보드 코드	dashboard/	Express 서버 + zero-token 라이브러리 (testQueue/tcUpload/gitOps 등) + 프론트엔드
slack-bridge/ 코드	slack-bridge/	Slack 봇 코드(비활성) — 평일 노션 정기보고 스크립트만 예외적으로 계속 가동

05 알려진 문제·제약

현재 구조를 점검하며 발견한 실제 이슈입니다. 심각도는 데이터 손실 가능성과 사용자 혼동 정도 기준입니다.

이슈	심각도	설명
TC 개별 삭제 기능 없음	Medium	대시보드에 프로젝트 전체 삭제는 있지만, TC 한 건만 지우는 기능은 없음(채팅에 직접 요청하는 방식만 가능) — 사용자 요청으로 확인된 갭, 다음 순번 작업 대상
"리포트 관리" 전용 기능 부재	Medium	실행 이력 목록·날짜별 스냅샷 나열 정도만 있고, 기간별 추이·요약을 생성/저장/공유하는 전용 리포트 기능은 없음

headed 브라우저 실행 옵션이 상시구동 환경에서 무의미	Minor	대시보드가 Windows 예약 작업(S4U)의 세션 0에서 실행되는데, 세션 0은 로그인 화면(세션 1)에 창을 띄울 수 없음(OS 레벨 격리) — 실측 확인 후 선택 시 안내 문구 추가
File System Access API가 HTTP+비-localhost에서 비활성화	Minor	TC 뷰어의 "JSON 파일 연결/직접저장"은 보안 컨텍스트(HTTPS 또는 localhost)가 필요 — HTTP로 전환한 이후 LAN IP 접속자는 다운로드 방식으로만 저장 가능(localhost 접속자는 영향 없음)
TC 업로드 후 자동 재생성된 뷰어에 최신 기능이 누락되는 사례 발견	Minor	SKILL.md에 표준 기능으로 명시된 "체크박스 선택 실행"이 실제 재생성 결과물에서 한 차례 빠진 사례 확인(2026-09-29) — 재발 여부 후속 확인 필요
여러 Claude Code 세션이 같은 대시보드 코드 파일을 동시 편집 가능	Minor	터미널/IDE 세션과 대시보드 채팅 패널이 서로 다른 세션으로 같은 파일(예: <code>server.js</code>)을 동시에 고칠 수 있어, 커밋 하나에 서로 무관한 두 작업이 섞이는 경우가 실제로 있었음 — 기능적 문제는 없었으나 커밋 단위 분리에 주의 필요
agents-config/slack-bridge 마버전관리	Resolved	2026-08-24 git subtree 병합으로 해소 — 이제 이 PC 디스크 장애가 나도 GitHub 원격에서 규칙·코드 전체 복구 가능

06 사용자 워크플로우

Slack 명령어는 더 이상 쓰지 않습니다. 아래는 실제로 쓰는 두 경로의 조작 방법입니다.

경로 A — 터미널/IDE 직접 세션

데모사이트 프로젝트 장바구니 TC 만들어

자연어 요청 — Phase 0(신규 프로젝트면)부터 순서대로 진행, 매 Phase 승인 필요

테스트 실행해줘

Phase 4 승인 후에만 유효 — 자동으로 실행되지 않고 반드시 별도 요청해야 Phase 5 시작

경로 B — 대시보드 조작

화면 요소

동작

채팅 패널에 메시지 입력	경로 A와 동일한 자연어 요청 (프로젝트 선택 상태에서)
테스트 실행 큐 → 필터 선택 → "▶ 테스트 실행 시작"	선택 조건에 맞는 TC를 순차 실행(zero-token)
"+ TC 업로드" → 파일 선택 → "업로드 및 반영"	엑셀/JSON을 캐노니컬 TC json에 즉시 반영 + 뷰어 재생성 자동 요청
결함 관리 표의 상태·담당자·이슈링크 셀 클릭	바로 편집, 저장 시 즉시 defects.json 반영
"🔗 Git" → "origin으로 push"	쌓인 커밋을 원격에 반영 (매번 직접 클릭해야 함)

07 한눈에 보는 용어

용어	의미
큐돌이	이 시스템 AI 어시스턴트의 페르소나 이름 — 과거엔 Slack 봇, 지금은 대시보드 채팅 패널(및 터미널/IDE 세션)에서 동일한 규칙으로 응답
zero-token	Claude를 호출하지 않고 대시보드 코드가 파일을 직접 읽고 쓰는 처리 방식 — 속도가 빠르고 비용이 안 들
Phase 0~8	온보딩 → 근거확보 → 범위산정 → 시나리오 → TC작성 → 실행 → 오류공유 → 결함관리 → 결과도출의 표준 워크플로우 (AGENTS.md 13항)
캐노니컬 TC 파일	<code>{프로젝트명}_TC_{모듈코드}.json</code> — 모듈당 하나, 실제 편집/버전관리 (<code>meta.version</code> / <code>changeHistory</code>)가 이뤄지는 단일 소스
테스트 실행 큐	대시보드에서 조건별(모듈/시스템/우선순위/수행상태)로 Playwright를 순차 실행하는 zero-token 기능
defects.json	프로젝트별 결함 대장 (자동/수동 등록, 상태 이력 포함)
<code>_template</code>	신규 프로젝트가 반드시 복사해서 시작해야 하는 빈 스캐폴드

